🎯 Adding New Object Detectors to AdvReal

> **Complete Integration Guide for Custom Detection Models**

This comprehensive guide walks you through integrating a new object detection model into the AdvReal adversarial patch generation framework. Whether you're adding YOLOv8, DETR variants, or any custom detector, this guide has you covered.

📋 Overview

Key Insight: AdvReal treats detectors as **black boxes** — you only need to:

1. ✅ Specify the input tensor size
2. ✅ Return standardized output format
3. ✅ No need to understand internal architecture details

🗺️ Architecture Map: Where Detectors Live

Understanding the codebase structure is crucial for successful integration:

Component	File	Purpose	Modification Required?
🔧 **Detector Registry**	`detlib/utils.py`	Model initialization hub	✅ **YES** - Add new detector
📦 **Base Interface**	`detlib/base.py`	Abstract detector contract	❌ No
⚔️ **Attack Loop**	`attack/methods/base.py`	Consumes detector outputs	❌ No
🎭 **Multi-Detector**	`attack/attacker.py`	Orchestrates multiple models	❌ No
🎨 **3D Training Path**	`train.py`, `load_data.py`	Patch optimization pipeline	⚠️ **Optional** - For train.py usage
🔄 **Output Conversion**	`utils_camou.py`	Format standardization	⚠️ **If needed** - For non-YOLO models

---

📤 Detector Output Contract

Your detector wrapper **must** return a dictionary with this exact structure:

```
{
    "bbox_array": List[Tensor[N, 6]],           # Per-image detections
                                                 # Format: [x1, y1, x2, y2, conf,
```

```
cls_id]
                                                    # ⚠ CRITICAL: Normalized to [0,
1]

    "obj_confs": Tensor[B, K],                    # Confidence scores for gradient
flow
                                                    # Shape: [batch_size,
num_detections]
                                                    # Device: Same as model output

    "cls_max_ids": Optional[Tensor[B, K]]         # Class indices (can be None)
                                                    # Required only for class-
specific attacks
}
```

🎯 Critical Requirements

Field	Requirement	Why It Matters
`bbox_array`	**Normalized [0,1]**	Ensures spatial consistency across image sizes
`obj_confs`	**Torch tensor** on correct device	Enables gradient-based optimization
`cls_max_ids`	Optional	Only needed for targeted class attacks

🚀 Integration Path 1: Multi-Detector Attack Framework

Use this path when: Running adversarial evaluations across multiple detectors

📝 Step A: Create the Wrapper Class

Location: `detlib/HHDet/<your_model>/api.py`

```
import torch
from ...base import DetectorBase

class HHMyDetector(DetectorBase):
    """
    Custom detector wrapper for [Your Model Name]

    This class adapts your detection model to AdvReal's standardized interface.
    """

    def __init__(self, name, cfg, input_tensor_size=640,
                 device=torch.device("cuda:0" if torch.cuda.is_available() else
"cpu")):
        """
        Initialize detector configuration

        Args:
```

```
            name (str): Detector identifier (e.g., "mydet")
            cfg: Configuration object with thresholds
            input_tensor_size (int): Square input size (e.g., 640 for 640×640)
            device (torch.device): Computation device
        """
        super().__init__(name, cfg, input_tensor_size, device)
        # Add model-specific initialization here
        self.custom_param = ...

    def load(self, model_weights, **kwargs):
        """
        Load pre-trained model weights

        Args:
            model_weights (str): Path to checkpoint file
            **kwargs: Additional model configuration
        """
        # Example: Load your model architecture
        self.detector = YourModelClass()
        self.detector.load_state_dict(torch.load(model_weights))
        self.detector.to(self.device)
        self.eval()  # Set to evaluation mode

    def __call__(self, batch_tensor, **kwargs):
        """
        Run inference and return standardized output

        Args:
            batch_tensor (torch.Tensor): Input images [B, C, H, W]

        Returns:
            dict: Standardized detection output
        """
        # 1 Forward pass through your model
        raw_output = self.detector(batch_tensor)

        # 2 Parse detections into bbox_array format
        bbox_array = []  # List of [N, 6] tensors per image
        for img_preds in raw_output:
            # Convert to [x1, y1, x2, y2, conf, cls_id]
            # ⚠ NORMALIZE coordinates to [0, 1]
            boxes = self._parse_predictions(img_preds)
            bbox_array.append(boxes)

        # 3 Extract confidence scores for gradient computation
        obj_confs = self._extract_confidences(raw_output)

        # 4 (Optional) Extract class IDs
        cls_max_ids = self._extract_classes(raw_output)

        return {
            "bbox_array": bbox_array,
            "obj_confs": obj_confs,
            "cls_max_ids": cls_max_ids  # or None
```

```
        }

    def _parse_predictions(self, predictions):
        """Helper: Convert model output to standardized box format"""
        # Implementation depends on your model's output structure
        pass

    def _extract_confidences(self, raw_output):
        """Helper: Extract objectness scores as differentiable tensor"""
        # Must return tensor with gradient tracking enabled
        pass
```

✒️ Step B: Register in Detector Factory

Location: `detlib/utils.py` → `init_detector()` function

```
def init_detector(detector_name: str, cfg: object, device: torch.device = ...):
    detector = None
    detector_name = detector_name.lower()  # ⚠ Always lowercase

    # ... existing detectors ...

    elif detector_name == "mydet":  # 👈 Add your detector here
        from detlib.HHDet import HHMyDetector
        detector = HHMyDetector(name=detector_name, cfg=cfg, device=device)

        # Load weights with appropriate paths
        model_weights = os.path.join(DET_LIB, 'HHDet/mydet/weights/best.pt')
        model_config = os.path.join(DET_LIB, 'HHDet/mydet/config/model.yaml')

        detector.load(
            model_weights=model_weights,
            model_config=model_config  # Optional
        )

    return detector
```

⚙️ Step C: Create Configuration File

Location: `configs/baseline/mydet.yaml`

```
# Dataset Configuration
DATA:
  CLASS_NAME_FILE: 'configs/namefiles/coco80.names'
  AUGMENT: 0

  TRAIN:
```

```yaml
    IMG_DIR: 'data/INRIAPerson/Train/pos'
    LAB_DIR: null

# Detector Settings
DETECTOR:
  NAME: ["MYDET"]              # 👈 Must match detector_name in utils.py
  INPUT_SIZE: [640, 640]       # Model input resolution
  BATCH_SIZE: 4                # Adjust based on GPU memory
  CONF_THRESH: 0.25            # Confidence threshold for NMS
  IOU_THRESH: 0.45             # IoU threshold for NMS

  PERTURB:
    GATE: null

# Attack Configuration
ATTACKER:
  METHOD: "pgd"                # Attack algorithm: pgd, bim, mim, optim
  EPSILON: 255                 # Max perturbation magnitude
  MAX_EPOCH: 1000
  ITER_STEP: 1
  STEP_LR: 0.03
  ATTACK_CLASS: '0'            # Target class ID (0=person in COCO)
  LOSS_FUNC: "obj-tv"
  tv_eta: 2.5                  # Total variation loss weight

  PATCH:
    WIDTH: 300
    HEIGHT: 300
    SCALE: 0.15                # Relative patch size
    INIT: "gray"               # Initialization: gray, random, white
    TRANSFORM: ['jitter', 'rotate', 'median_pool']
```

🗁 Step D: Organize Model Weights

Create the following directory structure:

```
detlib/HHDet/mydet/
├── api.py                     # Your wrapper class
├── __init__.py                # Export: from .api import HHMyDetector
├── weights/
│   └── best.pt              # Pre-trained checkpoint
└── config/
    └── model.yaml           # (Optional) Model architecture config
```

Weight file guidelines:

- ✅ Use relative paths in `detlib/utils.py`
- ✅ Keep weights inside project structure for portability
- ✅ Document download links in README if weights are large

Integration Path 2: 3D Patch Training Pipeline

Use this path when: Training adversarial patches with 3D rendering and mesh texturing

This path enables the full AdvReal pipeline including:

- Non-rigid surface modeling (NRSM)
- Realistic 3D mesh rendering
- Color transformation and relighting
- Physical world robustness

Step A: Add Model Branch in PatchTrainer

Location: `train.py` → `PatchTrainer.__init__()`

```
class PatchTrainer(object):
    def __init__(self, args):
        # ... existing initialization ...

        # 🎯 Add your detector loading logic here
        elif args.arch == "mydet":
            from your_model_package import MyDetectionModel

            # Load configuration
            cfg = ConfigParser("configs/baseline/mydet.yaml")
            detector_cfg = cfg.DETECTOR

            # Initialize model
            self.model = MyDetectionModel(
                input_size=self.img_size,
                device=self.device
            )

            # Load pre-trained weights
            weights_path = 'path/to/mydet_weights.pt'
            self.model.load_weights(weights_path)

            # Set to evaluation mode and freeze parameters
            self.model.eval()
            for param in self.model.parameters():
                param.requires_grad = False  # ⚠️ Critical: Don't update detector
weights
```

Step B: Create Probability Extractor

Location: `load_data.py`

The probability extractor converts model outputs to adversarial loss signals.

```python
class MyDetMaxProbExtractor(nn.Module):
    """
    Extracts detection probabilities for adversarial loss computation

    This class bridges your detector's output format with AdvReal's loss
functions.
    """

    def __init__(self, cls_id, num_cls, model, figsize):
        """
        Args:
            cls_id (int): Target class ID to attack
            num_cls (int): Total number of classes in dataset
            model: The detector model instance
            figsize (int): Image size for coordinate scaling
        """
        super(MyDetMaxProbExtractor, self).__init__()
        self.cls_id = cls_id
        self.num_cls = num_cls
        self.figsize = figsize
        self.model = model
        self.cfg = None  # Will be set externally

    def forward(self, model_output, ground_truth, loss_type, iou_thresh):
        """
        Extract detection scores for adversarial optimization

        Args:
            model_output: Raw output from your detector
            ground_truth (Tensor): GT bounding boxes [B, 4] in pixel coords
            loss_type (str): 'max_iou', 'max_conf', 'softplus_max', etc.
            iou_thresh (float): IoU threshold for positive matches

        Returns:
            det_loss (Tensor): Scalar loss value for backpropagation
            max_probs (Tensor): Detection confidence scores
        """
        det_loss = []
        max_probs = []

        # [1] Parse model output into standardized box format
        # Format: [x_center, y_center, width, height, obj_conf, cls_conf, cls_id]
        box_all = self._parse_output(model_output)

        # [2] Process each image in the batch
        for i, boxes in enumerate(box_all):
            if boxes.numel() == 0:
                # No detections → append zero loss
                det_loss.append(torch.tensor(0.0, device=ground_truth.device))
                max_probs.append(torch.tensor(0.0, device=ground_truth.device))
                continue
```

```
            # [3] Convert center format to corner format [x1, y1, x2, y2]
            x_center, y_center, w, h = boxes[..., 0], boxes[..., 1], boxes[...,
2], boxes[..., 3]
            x1 = x_center - w / 2
            y1 = y_center - h / 2
            x2 = x_center + w / 2
            y2 = y_center + h / 2
            bbox = torch.stack([x1, y1, x2, y2], dim=-1)

            # [4] Compute IoU with ground truth boxes
            ious = torchvision.ops.box_iou(
                bbox.view(-1, 4) * self.figsize,  # Scale to pixel coordinates
                ground_truth[i].unsqueeze(0)
            ).squeeze(-1)

            # [5] Filter by IoU threshold and target class
            conf = boxes[..., 4]
            cls_idx = boxes[..., 6].long()

            # Read attack class from config (like YOLOv11 pattern)
            attack_cls = int(getattr(self.cfg, 'ATTACKER',
EasyDict(ATTACK_CLASS='0')).ATTACK_CLASS)

            mask = ious.ge(iou_thresh) & (cls_idx == attack_cls)
            valid_ious = ious[mask]
            valid_scores = conf[mask]

            # [6] Compute loss based on selected strategy
            if valid_scores.numel() > 0:
                if loss_type == 'max_iou':
                    # Use detection with maximum IoU
                    det_loss.append(valid_scores.max())
                    max_probs.append(valid_scores.max())

                elif loss_type == 'max_conf':
                    # Use maximum confidence score
                    det_loss.append(valid_scores.max())
                    max_probs.append(valid_scores.max())

                elif loss_type == 'softplus_max':
                    # Smooth maximum using softplus
                    max_conf = F.softplus(-torch.log(1.0 / valid_scores.max() -
1.0))
                    det_loss.append(max_conf)
                    max_probs.append(valid_scores.max())

                # Add more loss types as needed...

            else:
                # No valid detections matching criteria
                det_loss.append(ious.new([0.0])[0])
                max_probs.append(ious.new([0.0])[0])
```

```
        # 7 Aggregate losses across batch
        det_loss = torch.stack(det_loss).mean()
        max_probs = torch.stack(max_probs)

        return det_loss, max_probs

    def _parse_output(self, model_output):
        """
        Convert your model's output to standardized format

        Returns:
            List[Tensor]: Each tensor is [N, 7] with columns:
                          [x_center, y_center, w, h, obj_conf, cls_conf, cls_id]
        """
        # Implementation depends on your specific model output format
        # See utils_camou.get_region_boxes_general for examples
        pass
```

Step C: Extend Output Conversion (If Needed)

Location: `utils_camou.py` → `get_region_boxes_general()`

If your model has a unique output format, add a conversion branch:

```
def get_region_boxes_general(output, model, conf_thresh=0.5, name="custom"):
    """
    Universal box format converter for different detector architectures

    Args:
        output: Raw model predictions
        model: Detector model instance
        conf_thresh (float): Confidence threshold
        name (str): Detector identifier

    Returns:
        List[Tensor]: Standardized boxes [N, 7] format
    """

    if name == "mydet":
        # 👈 Add your custom parsing logic here
        boxes_list = []

        for batch_idx in range(output.shape[0]):
            # Parse your model's specific output structure
            # Example: If output is [B, num_anchors, 85] (YOLO-style)
            predictions = output[batch_idx]

            # Extract box coordinates (scale to [0, 1])
            x_center = predictions[:, 0]
            y_center = predictions[:, 1]
            width = predictions[:, 2]
```

```
            height = predictions[:, 3]

            # Extract objectness and class scores
            obj_conf = predictions[:, 4]
            cls_scores = predictions[:, 5:]
            cls_conf, cls_id = cls_scores.max(dim=-1)

            # Filter by confidence threshold
            mask = obj_conf > conf_thresh

            # Stack into [N, 7] format
            boxes = torch.stack([
                x_center[mask],
                y_center[mask],
                width[mask],
                height[mask],
                obj_conf[mask],
                cls_conf[mask],
                cls_id[mask].float()
            ], dim=-1)

            boxes_list.append(boxes)

        return boxes_list

    elif name == "yolov2":
        # Existing YOLO implementations...
        pass

    # ... other detectors ...
```

✒️ Step D: Wire Extractor in Training Script

Location: `train.py` → `PatchTrainer.__init__()`

```
# After model loading, create the appropriate probability extractor
if args.arch == "mydet":
    from load_data import MyDetMaxProbExtractor

    self.prob_extractor = MyDetMaxProbExtractor(
        cls_id=0,           # Will be overridden by cfg
        num_cls=80,         # Number of classes in your dataset
        model=self.model,
        figsize=self.img_size
    )

    # ⚠ Critical: Link config so extractor can read ATTACK_CLASS
    self.prob_extractor.cfg = cfg
```

💡 Complete Example: YOLOv11 Integration

Let's walk through the actual YOLOv11 integration as a complete reference.

🏗️ Architecture Overview

```
YOLOv11 Integration
│
├── 📄 Wrapper Class
│   └── detlib/HHDet/yolov11/api.py
│       └── class HHYolov11(DetectorBase)
│
├── ✒️ Registry Entry
│   └── detlib/utils.py
│       └── init_detector() → "yolov11" branch
│
├── ⚙️ Configuration
│   └── configs/baseline/v11.yaml
│       └── DETECTOR.NAME: ["YOLOV11"]
│
├── 🗂️ Model Weights
│   └── detlib/HHDet/yolov11/weights/
│       └── yolo11s.pt
│
└── 🎨 Training Pipeline (Optional)
    ├── train.py → args.arch == "yolov11"
    ├── load_data.py → YOLOv11MaxProbExtractor
    └── utils_camou.py → get_region_boxes_general("yolov11")
```

📝 Step 1: Wrapper Implementation

File: `detlib/HHDet/yolov11/api.py`

```
import torch
import numpy as np
from ultralytics.utils.ops import non_max_suppression as ul_nms
from ...base import DetectorBase

class HHYolov11(DetectorBase):
    """YOLOv11 detector wrapper with robust NMS handling"""

    def __init__(self, name, cfg, input_tensor_size=640,
                 device=torch.device("cuda:0" if torch.cuda.is_available() else
"cpu")):
        super().__init__(name, cfg, input_tensor_size, device)
        self.imgsz = (input_tensor_size, input_tensor_size)
        self.conf_thres = getattr(self, 'conf_thres', 0.25)
        self.iou_thres = getattr(self, 'iou_thres', 0.45)
```

```
    def load(self, model_weights, **args):
        from ultralytics import YOLO
        yolo_wrapper = YOLO(model_weights)
        self.detector = yolo_wrapper.model.to(self.device)
        self.detector.eval()
        self.names = self.detector.names

    def __call__(self, batch_tensor, **kwargs):
        batch_tensor = batch_tensor.to(self.device)
        B, _, H, W = batch_tensor.shape

        # 1 Forward pass: [B, 84, 8400] for 80 classes
        preds = self.detector(batch_tensor)
        if isinstance(preds, (list, tuple)):
            preds = preds[0]

        # 2 Extract objectness for gradient flow
        preds_transposed = preds.transpose(-1, -2)  # [B, 8400, 84]
        pred_cls_logits = preds_transposed[..., 4:]
        obj_confs = torch.sigmoid(pred_cls_logits).max(dim=-1).values

        # 3 Apply NMS (CRITICAL: use untransposed tensor!)
        nms_preds = ul_nms(preds, self.conf_thres, self.iou_thres,
multi_label=False)

        # 4 Normalize bounding boxes
        bbox_array = []
        for det in nms_preds:
            if det is None or len(det) == 0:
                bbox_array.append(torch.empty((0, 6), device=self.device))
                continue

            det_norm = det.clone()
            if det_norm[:, :4].max() > 1.0:  # If in pixel coordinates
                det_norm[:, [0, 2]] /= W
                det_norm[:, [1, 3]] /= H
            bbox_array.append(det_norm)

        return {
            'bbox_array': bbox_array,
            'obj_confs': obj_confs,
            'cls_max_ids': None
        }
```

🔑 **Key Points:**

- Uses Ultralytics API but extracts raw PyTorch model
- Handles NMS with proper tensor dimensions
- Normalizes coordinates to [0, 1] range
- Maintains gradient flow through `obj_confs`

✒️ Step 2: Registry Entry

File: detlib/utils.py

```python
def init_detector(detector_name: str, cfg: object, device=...):
    detector = None
    detector_name = detector_name.lower()  # ⚠️ Convert to lowercase

    # ... other detectors ...

    elif detector_name == "yolov11":
        from detlib.HHDet import HHYolov11
        detector = HHYolov11(name=detector_name, cfg=cfg, device=device)

        weights_path = os.path.join(DET_LIB, 'HHDet/yolov11/weights/yolo11s.pt')
        detector.load(model_weights=weights_path)

    return detector
```

⚙️ Step 3: Configuration File

File: configs/baseline/v11.yaml

```yaml
DATA:
  CLASS_NAME_FILE: 'configs/namefiles/coco80.names'
  AUGMENT: 0
  TRAIN:
    IMG_DIR: 'data/INRIAPerson/Train/pos'

DETECTOR:
  NAME: ["YOLOV11"]      # 👈 Uppercase in config
  INPUT_SIZE: [416, 416]
  BATCH_SIZE: 8
  CONF_THRESH: 0.5
  IOU_THRESH: 0.45

ATTACKER:
  METHOD: "optim"
  EPSILON: 255
  ATTACK_CLASS: '0'      # Person class
  LOSS_FUNC: "obj-tv"
  tv_eta: 2.5

  PATCH:
    WIDTH: 300
    HEIGHT: 300
    INIT: "gray"
```

🗂 Step 4: Directory Structure

```
detlib/HHDet/yolov11/
├── __init__.py           # from .api import HHYolov11
├── api.py                # HHYolov11 class implementation
└── weights/
    └── yolo11s.pt       # Download from Ultralytics
```

🧪 Step 5: Testing Your Integration

```python
# Quick sanity check script
import torch
from utils.parser import ConfigParser
from detlib.utils import init_detector

# Load config
cfg = ConfigParser("configs/baseline/v11.yaml")

# Initialize detector
detector = init_detector("yolov11", cfg.DETECTOR)

# Create dummy input
batch = torch.rand(2, 3, 416, 416).cuda()

# Run inference
output = detector(batch)

# Verify output format
assert 'bbox_array' in output
assert 'obj_confs' in output
assert len(output['bbox_array']) == 2  # Batch size

# Check normalization
for boxes in output['bbox_array']:
    if boxes.numel() > 0:
        assert boxes[:, :4].max() <= 1.0, "Boxes not normalized!"
        assert boxes[:, :4].min() >= 0.0, "Negative coordinates!"

print("✅ Integration successful!")
```

✅ Validation Checklist

Before deploying your detector integration, verify these critical points:

🎯 Output Format Validation

- [ ] **Bounding boxes normalized** to [0, 1] range
- [ ] **bbox_array** is a list of tensors (one per batch image)
- [ ] **obj_confs** is a torch.Tensor on correct device
- [ ] **Gradient flow** enabled through obj_confs
- [ ] **Device consistency** (all tensors on same GPU/CPU)

⚙️ Configuration Validation

- [ ] **Detector name** matches in:
  - [ ] Config file: `DETECTOR.NAME: ["MYDET"]` (uppercase)
  - [ ] Registry: `detector_name == "mydet"` (lowercase)
- [ ] **Weights path** is correct and accessible
- [ ] **Class names file** exists and contains correct labels
- [ ] **Input size** matches model requirements

🧪 Functional Testing

- [ ] **Forward pass** completes without errors
- [ ] **Batch inference** works (B > 1)
- [ ] **Empty predictions** handled gracefully (no crashes on blank images)
- [ ] **Gradient tracking** preserved through detection pipeline
- [ ] **Memory cleanup** (no leaks during extended runs)

📊 Performance Validation

Run this test to measure inference speed:

```
import time
import torch
from detlib.utils import init_detector
from utils.parser import ConfigParser

cfg = ConfigParser("configs/baseline/mydet.yaml")
detector = init_detector("mydet", cfg.DETECTOR)

# Warmup
dummy = torch.rand(4, 3, 416, 416).cuda()
for _ in range(10):
    _ = detector(dummy)

# Benchmark
times = []
for _ in range(100):
    start = time.time()
    output = detector(dummy)
    torch.cuda.synchronize()
    times.append(time.time() - start)

print(f"⚡ Average inference time: {sum(times)/len(times)*1000:.2f} ms")
print(f"📊 FPS: {1/(sum(times)/len(times)):.1f}")
```

⚠ Common Pitfalls & Solutions

Issue	Symptom	Solution
🔴 **Coordinates not normalized**	Boxes at [413.5, 227.8, ...]	Divide by image width/height
🔴 **Wrong device**	CUDA/CPU mismatch errors	Use `.to(self.device)` consistently
🔴 **List instead of tensor**	`obj_confs` gradient errors	Use `torch.stack()` or `torch.cat()`
🔴 **No gradient flow**	Loss doesn't change during attack	Ensure `requires_grad=True` in forward pass
🔴 **Name mismatch**	`KeyError: 'mydet'`	Check uppercase (config) vs lowercase (code)
🔴 **Missing NMS**	Too many duplicate boxes	Apply `torchvision.ops.nms()` or equivalent
🔴 **Empty batch handling**	Crashes on no detections	Return empty tensors with correct shape

🗄 Reference Implementation Files

Use these as templates when integrating your detector:

🏆 Recommended References

Model Type	Reference File	Key Features
YOLO-style	`detlib/HHDet/yolov11/api.py`	NMS handling, gradient flow
Two-stage	`detlib/torchDet/faster_rcnn.py`	ROI extraction, class filtering
Transformer	`detlib/HHDet/ddetr/api.py`	Attention-based detection

🗁 Key Files to Study

```
🗁 detlib/
├── base.py              # Abstract detector interface
├── utils.py             # Detector registry and initialization
├── HHDet/
│   ├── yolov5/api.py   # Classic YOLO implementation
│   ├── yolov11/api.py  # Modern YOLO with Ultralytics
│   └── ddetr/api.py    # Transformer-based detector
└── torchDet/
    └── faster_rcnn.py  # Two-stage detector example

🗁 attack/
├── attacker.py          # How detectors are used in attacks
```

```
└── methods/base.py      # Loss computation from detector outputs

📁 utils/
└── det_utils.py         # Bounding box utilities (IoU, NMS, etc.)

📁 load_data.py          # Probability extractors for training
```

🚀 Quick Start Command

After completing integration, test with:

```
# Multi-detector attack evaluation
python scripts/demo.py \
    --cfg configs/baseline/mydet.yaml \
    --patch results/universal_patch.png \
    --input_dir data/test_images/

# Full 3D patch training pipeline
python train.py \
    --nepoch 800 \
    --save_path results/mydet_patch \
    --arch mydet \
    --cfg configs/baseline/mydet.yaml \
    --seed_type fixed \
    --loss_type max_iou
```

💬 Need Help?

If you encounter issues:

1. 📖 **Check existing implementations** in `detlib/HHDet/` for similar detector types
2. 🔍 **Search error messages** in `attack/methods/base.py` for clues
3. 🔦 **Run validation checklist** above to isolate the problem
4. 📊 **Add debug prints** in your `__call__()` method to inspect tensor shapes

---

📝 Summary

Minimum viable integration:

1. ✅ Create wrapper class (`detlib/HHDet/mydet/api.py`)
2. ✅ Register in factory (`detlib/utils.py`)
3. ✅ Add config file (`configs/baseline/mydet.yaml`)
4. ✅ Return standardized output (bbox_array, obj_confs)

For full training support: 5. ✅ Add probability extractor (`load_data.py`) 6. ✅ Wire in training script (`train.py`) 7. ✅ (Optional) Add output converter (`utils_camou.py`)

You're ready to integrate any object detector into AdvReal!